



Primary Bottlenecks That Hurt the Scalability of our Application

We'll cover the following



- Database
- Application design
- Not using caching in the application wisely
- Inefficient configuration and setup of load balancers
- Adding business logic to the database
- Not picking the right database
- At the code level

There are several points in a web application that can become a *bottleneck* and hurt the *scalability* of our application. Let's take a look at them.

Database

Imagine we have an application that appears to be well architected. Everything looks good. The workload runs on multiple nodes, and it can scale horizontally.

However, the *database* is a poor single monolith, where just one server has the onus of handling the data requests from all the server nodes of the workload.





This scenario is a bottleneck. The server nodes work well, handle millions of requests at a point in time efficiently, yet, the response time of these requests and the latency of the application are abysmal due to the presence of a single database. There is only so much it can handle.

Just like workload scalability, the database needs to be scaled well.

Make wise use of *database partitioning*, *sharding* with multiple database servers to make your system efficient.

Application design

A poorly designed application's architecture can become a major bottleneck as a whole.

A typical architectural mistake is not using *asynchronous processes* and modules wherever required; rather, all the processes are scheduled sequentially.

For example, if a user uploads a document on the portal, tasks such as sending a confirmation email to the user, sending a notification to all



Tasks like these should be forwarded to a *messaging server* or a *task queue* for asynchronous processing as opposed to being processed sequentially, making the user wait.

Not using caching in the application wisely

Caching can be deployed at several layers of the application. It speeds up the response time by notches. A *cache* cuts down the overall load on the app, intercepting all the requests before they hit the origin servers.

We should use *caching* exhaustively throughout the application to speed up things significantly.

If the system has a lot of static data, caching can bring down the deployment costs significantly. I've written an article on my blog: [How PolyHaven manages 5 million page views and 80TB traffic a month for less than 400 USD](#).



Polyhaven is a 3D asset library with a large amount of static data. The article delineates how it leverages caching to bring down its deployment costs.

Inefficient configuration and setup of load balancers

Load balancers are the gateway to our application. Using too many or too few of them impacts the latency of our application. More on load balancers in the upcoming lessons.

Adding business logic to the database

No matter what justification anyone provides, I've never been a fan of adding *business logic* to the *database*.

The database is just not the place to put business logic. Business logic in the database makes the application components tightly coupled. Imagine how much code refactoring this would require when migrating to a different database. Also, the testing gets complex.

Not picking the right database

Picking the right database technology is vital for businesses. Need *transactions* and *strong consistency*? Pick a *relational database*. If you can do without *strong consistency* and rather need *horizontal scalability*, pick a *NoSQL* database.

Trying to pull things off with a not-so-suitable tech always has a profound impact on the latency of the entire application in negative ways. More on this in the upcoming lessons.



At the code level

This shouldn't come as a surprise, but inefficient and poorly written code has the potential to bring down the entire service in production. This typically includes:

- Using unnecessary loops or nested loops
- Writing tightly coupled code
- Not paying attention to the *Big-O* complexity while writing the code. (Be ready to do a lot of firefighting in production)

Ideally, we should always do a *DENTAL* (Documentation, Exception Handling, Null pointers, Time complexity, Test coverage, Analysis of code complexity, Logging) check of our code when doing a dry run.

In this lesson, don't worry if a few things are not clear to you, such as *strong consistency*, how the *message queue* facilitates *asynchronous behavior*, or how to pick the right database. I'll discuss all that in the upcoming lessons. Stay tuned.

Let's move on to the next lesson.

[< Back](#)[Mark As Completed](#)[Next >](#)

Which Scalability Approach is Right for...

How to Improve and Test the Scalabilit...

